



L14: Drone Pathfinding

Linear Algorithms & Navigation

Neon City Cyber Academy

Module 4: Visual Algorithms





Your Team

Emily Chen - *Handler*

"Forget drawing turtles. Today you're programming autonomous drones to navigate the city grid."

Cole Martinez - *Tech Lead*

"Every screen is a coordinate system. Master X and Y, and you control any pixel, any drone, any game object."

Module Overview: Drone Pathfinding

What we're NOT doing: Drawing cute turtle shapes

What we ARE doing: Programming navigation algorithms

Real-World Applications:

-  Autonomous drone delivery
-  Video game character movement
-  Robot pathfinding
-  Data visualization plotting

The screen is a battlefield. You control it with coordinates.



The Cartesian Coordinate System



+X

+Y

-Y

Understanding Coordinates

Every point on screen has an address: (x, y)

```
# Drone position  
x = 5    # 5 units RIGHT from origin  
y = 3    # 3 units UP from origin
```



Emily's Coordinate Rules:

"Think of coordinates like a treasure map:"

- **X (horizontal)**: Positive = Right, Negative = Left
- **Y (vertical)**: Positive = Up, Negative = Down

"Moving RIGHT increases X. Moving UP increases Y."

```
position = (10, 20)
# X = 10 (10 units right of origin)
# Y = 20 (20 units above origin)
```



Drone Commands: Linear Movement

```
# Start at origin
x = 0
y = 0

# Move RIGHT 10 units
x = x + 10    # Now at (10, 0)

# Move UP 5 units
y = y + 5     # Now at (10, 5)

# Move LEFT 3 units
x = x - 3     # Now at (7, 5)

print(f"Drone position: ({x}, {y})")
```

Output: Drone position: (7, 5)



Visualizing Movement

(0, 0) START (10, 0) (10, 5) (7, 5) END



Cole's Navigation Insight:

"Every movement is just MATH:"

```
# Forward (direction matters!)  
x += distance # If facing right  
y += distance # If facing up  
  
# The drone doesn't 'know' directions  
# It only knows: change X, change Y
```

"In advanced systems, we add a 'heading' (angle). But today:
pure coordinates."



Calculating Distance

How far between two points?

Pythagorean Theorem: $\text{distance} = \sqrt{(\Delta x^2 + \Delta y^2)}$

```
import math

x1, y1 = 0, 0    # Start
x2, y2 = 3, 4    # End

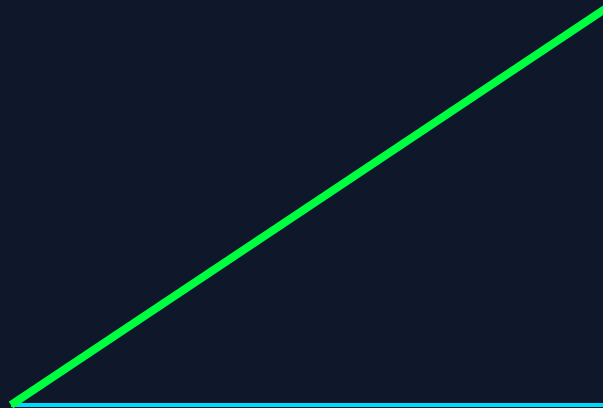
delta_x = x2 - x1  # 3
delta_y = y2 - y1  # 4

distance = math.sqrt(delta_x**2 + delta_y**2)
print(distance)    # 5.0
```

3-4-5 triangle: Classic example!



Distance Visualization



$$\Delta x = 3$$

$$\Delta y = 4$$

$$\text{distance} = 5$$

12 34 Example: Delivery Route

```
# Drone starts at warehouse
warehouse_x, warehouse_y = 0, 0

# Customer location
customer_x, customer_y = 6, 8

# Calculate delivery distance
dx = customer_x - warehouse_x # 6
dy = customer_y - warehouse_y # 8

distance = (dx**2 + dy**2) ** 0.5 # sqrt
print(f"Delivery distance: {distance:.2f} km")
```

Output: Delivery distance: 10.00 km

Emily's Real-World Context:

"GPS uses coordinates constantly:"

- Your phone: (Latitude, Longitude) = (X, Y)
- Google Maps: Calculates distance between points
- Uber: Finds nearest driver using distance formula

"Every 'Find Near Me' feature? **Coordinate math.**"



Game Development Application

```
# Player position
player_x = 10
player_y = 15

# Enemy position
enemy_x = 13
enemy_y = 19

# Check if enemy is in range
attack_range = 5
distance = ((enemy_x - player_x)**2 +
            (enemy_y - player_y)**2) ** 0.5

if distance <= attack_range:
    print("Enemy in range! FIRE!")
else:
    print("Too far. Move closer.")
```



Drone Simulation (Text-Based)

```
# Drone flight path
x, y = 0, 0
path = []

# Mission waypoints
x, y = x + 5, y + 0
path.append((x, y))

x, y = x + 0, y + 3
path.append((x, y))

x, y = x - 2, y + 2
path.append((x, y))

print("Flight path:")
for i, pos in enumerate(path, 1):
    print(f"Waypoint {i}: {pos}")
```



Output Format

Flight path:

Waypoint 1: (5, 0)

Waypoint 2: (5, 3)

Waypoint 3: (3, 5)

This is what Judge0 will test: Text output, not graphics!

✓ Lesson Complete!

You Mastered:

- ✓ Cartesian coordinate system (X, Y)
- ✓ Origin point (0, 0)
- ✓ Positive/negative directions
- ✓ Linear movement (changing X and Y)
- ✓ Distance calculation (Pythagorean theorem)
- ✓ Flight path simulation

Next Mission: L15 - Loops in Graphics

Pattern Recognition: Drawing shapes efficiently



Your Assignment

Create a **Patrol Route Calculator**:

1. Define 4 waypoints as (x, y) tuples
2. Calculate distance between each consecutive pair
3. Print total patrol distance
4. Use `math.sqrt()` for accuracy

Hint: Loop through waypoints with `for i in range(len(points)-1)`